

Analytics Engineering: Session 11

Model Governance: Contracts, Versions & Access

Today

- Move from "models that build" to **models others can safely depend on**
- Use contracts to define model shape
- Use versions to manage breaking changes
- Use access levels to protect internal implementation details
- Practice with examples from this ecommerce dbt project

Governance In This Repo

The project already has a layered contract, even before explicit model contracts.

- **Sources:** 14 raw DuckDB tables loaded from Parquet
- **Staging:** thin `stg_*` models over raw data
- **Intermediate:** joins and business logic in `int_*`
- **Marts:** reporting models like `dim_customers` and `mart_orders`

Governance makes those boundaries explicit.

Why Model Governance?

As dbt projects grow, teams need rules to prevent downstream breakage.

- **Contracts:** Guarantee the shape of your data
- **Versions:** Allow breaking changes without breaking consumers immediately
- **Access:** Control who can reference which models

Governance = Contracts + Versions + Access

Existing Example: `dim_customers`

The model already documents its public shape:

```
models:
- name: dim_customers
  description: Customer dimension...
  columns:
    - name: customer_id
      tests: [unique, not_null]
    - name: email
      tests: [unique, not_null]
    - name: customer_segment
      tests:
        - accepted_values:
            values: ['Bronze', 'Silver', 'Gold', 'Platinum']
```

This is governance material waiting to become a stronger contract.

Contracts vs Tests

Contracts check the model interface

- Does the output have the declared columns?
- Do column data types match the YAML?
- Are enforced constraints part of the built relation?
- Fails while dbt is building the model

Tests check the rows inside it

- Is `customer_id` unique and not null?
- Does every order link to `dim_customers`?
- Are statuses only `completed`, `cancelled`, etc.?
- Runs after the model exists

Short version: contracts protect the table shape; tests protect the data quality.

Contract Example: `dim_customers`

Start with the columns consumers rely on most:

```
models:
- name: dim_customers
  access: public
  config:
    contract: {enforced: true}
  columns:
    - name: customer_id
      data_type: integer
      constraints:
        - type: not_null
    - name: email
      data_type: varchar
    - name: is_repeat_customer
      data_type: boolean
```

If SQL stops returning `email`, the model build fails.

Constraint Examples

Constraint	Good fit in this project
not_null	customer_id, order_id, status
unique	dim_customers.customer_id
primary_key	mart_orders.order_id
check	total_amount $\geq$ 0

```
- name: total_amount
  data_type: numeric
  constraints:
    - type: check
      expression: "total_amount  $\geq$  0"
```


Test Example: mart_orders

`mart_orders.yml` already declares a core referential rule:

```
- name: customer_id
  description: Foreign key to dim_customers.
  tests:
    - not_null
    - relationships:
        to: ref('dim_customers')
        field: customer_id
```

This says every order must belong to a known customer.

Business Domain Example: Order Status

The project uses the same status domain in staging and marts:

```
- name: status
description: Fulfillment status of the order.
tests:
  - not_null
  - accepted_values:
      values:
        - pending
        - shipped
        - completed
        - cancelled
        - refunded
```

This is a lightweight business contract.

Custom Test Example: Positive Amounts

`stg_orders.yml` uses the project macro `test_is_positive`:

```
- name: total_amount
  description: Final amount charged to the customer.
  tests:
    - not_null
    - is_positive:
      where: "status in ('completed', 'shipped')"
```

The rule is conditional: drafts and cancelled orders may behave differently.

Model Versions

Versions let consumers choose when to migrate:

```
models:
  - name: dim_customers
    latest_version: 2
    versions:
      - v: 1
        columns:
          - include: all
      - v: 2
        columns:
          - include: all
          - name: loyalty_tier
            data_type: varchar
```

Consumers can still call `{{ ref('dim_customers', v=1) }}`.

Versioning Example: Customer Dimension

Version 1

```
select
  customer_id,
  full_name,
  email,
  customer_segment,
  total_orders,
  total_spent
from final
```

Version 2

```
select
  customer_id,
  full_name,
  email,
  customer_segment,
  segment_id,
  average_order_value,
  is_repeat_customer
from final
```

Adding columns is usually safe; changing expectations may still need coordination.

Breaking Change Example: `mart_orders`

`mart_orders` is one row per order today.

Breaking changes would include:

- Renaming `total_amount` to `order_total`
- Removing `payment_status_summary`
- Changing `order_date` from date to timestamp
- Changing grain from **one row per order** to **one row per order item**

If the grain changes, every downstream metric can change.

Deprecation

Use deprecation to make migration visible:

```
models:
  - name: mart_orders
    latest_version: 2
    versions:
      - v: 1
        deprecation_date: 2026-12-31
      - v: 2
```

- dbt warns users referencing v1
- Teams get a concrete migration deadline
- Deprecation communicates; it does not redesign the model for you

Model Access

Control which models can be referenced by other projects or groups.

Level	Who can <code>ref()</code> it?	Good use
public	Any project or group	<code>dim_customers</code> , <code>mart_orders</code>
protected	Same project only	default for course models
private	Same group only	internal intermediate logic

Access is about dependency boundaries, not data security.

Access Example For This DAG

Expose marts. Hide implementation details.

```
models:  
  - name: dim_customers  
    access: public  
  
  - name: mart_orders  
    access: public  
  
  - name: int_orders_enriched  
    access: private  
  
  - name: int_payments_by_order  
    access: private
```

Dashboards depend on marts, not intermediate plumbing.

Why Hide Intermediate Models?

`int_orders_enriched` joins:

- order item summaries
- shipping status
- payment status
- generated order flags

It produces convenience fields:

```
is_completed_order,  
is_cancelled_order,  
is_refunded_order,  
is_paid
```

Consumers should depend on the stable mart, not this implementation layer.

Existing Logic: Shipping Buckets

`int_order_shipping_status` turns dates into a business label:

```
case
  when shipping.ship_date is null then 'not_shipped'
  when shipping.actual_delivery < shipping.estimated_delivery then 'early'
  when shipping.actual_delivery = shipping.estimated_delivery then 'on_time'
  when shipping.actual_delivery > shipping.estimated_delivery then 'late'
  else 'unknown'
end as shipping_performance_bucket
```

That label is governed with an `accepted_values` test.

Existing Logic: Payment Summary

`int_payments_by_order` defines the payment contract for marts:

```
case
  when count(payment_id) = 0 then 'unknown'
  when sum(case when status = 'completed' then 1 else 0 end)
    = count(payment_id) then 'paid'
  when sum(case when status = 'completed' then 1 else 0 end) > 0
    then 'partially_paid'
  else 'unpaid'
end as payment_status_summary
```

Governed values: `paid`, `partially_paid`, `unpaid`, `unknown`.

Python Model Governance: RFM

`mart_customer_rfm.py` creates scored customer segments:

```
def classify_segment(score):  
    if score ≥ 13:  
        return "champion"  
    elif score ≥ 10:  
        return "loyal"  
    elif score ≥ 7:  
        return "at_risk"  
    else:  
        return "lost"
```

The YAML test protects the output domain for `segment`.

Breaking vs Non-breaking Changes

Change	Example from this project	Action
Add a column	add <code>loyalty_tier</code> to <code>dim_customers</code>	usually safe
Add a test	require <code>status</code> not null	safe but may fail build
Rename a column	<code>total_amount</code> → <code>order_total</code>	version
Remove a column	drop <code>email</code> from <code>dim_customers</code>	version
Change grain	order grain → order item grain	version

Putting It All Together

Govern a real mart model with contract, version, and access:

```
models:
- name: mart_orders
  access: public
  latest_version: 1
  config:
    contract: {enforced: true}
  columns:
    - name: order_id
      data_type: integer
      constraints:
        - type: not_null
    - name: customer_id
      data_type: integer
      constraints:
        - type: not_null
  tests:
    - relationships:
        to: ref('dim_customers')
        field: customer_id
- name: payment_status_summary
  data_type: varchar
  tests:
    - accepted_values:
        values: ['paid', 'partially_paid', 'unpaid', 'unknown']
```

In-class Walkthrough

For any mart model, ask:

1. What is the grain?
2. What columns are part of the public API?
3. Which columns need `not_null`, `unique`, or relationship tests?
4. Which business labels need `accepted_values` ?
5. Would a proposed change break downstream consumers?
6. Should consumers reference this model directly?

What Have We Learned?

- Governance turns model ownership into explicit rules
- Contracts protect shape; tests protect values
- Versions let teams ship breaking changes responsibly
- Access levels protect project boundaries
- Existing models already contain strong governance examples

Next Session: Practice Session III: Production-Ready Data Quality.